A CASE-STUDY BASED REPORT

ON

Insurance Claim Fraud Detection using XG Boost

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Uppu Sindhuja **2520080045**

P. Bhagya Sri **2520080024**

Under the Esteemed guidance of

Ch. Anitha

Asst. Professor, Dept. of CS&IT

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

Insurance claim fraud can increase financial losses for insurers and create additional review work. This project presents SafeClaim, a student prototype that uses machine learning to estimate the probability that a vehicle insurance claim belongs to the fraudulent class. It uses the fraud_oracle.csv dataset and a saved XGBoost model. A Flask web application accepts claim details and displays the model's classification and estimated fraud probability. The application also displays separate, rule-based review indicators.

The dataset contains 15,420 records and 33 columns, including the target column. It contains 923 records labeled as fraudulent and 14,497 labeled as genuine. The project's performance page reports an accuracy of 95.43%, precision of 82.56%, recall of 36.04%, F1 score of 50.18%, and ROC-AUC of 67.76% for the selected Bayesian-optimized XGBoost model. The low recall is an important limitation: many fraud-labeled examples may not be identified by the model.

SafeClaim is an educational prototype. Its output is a model estimate and should not be treated as proof of fraud or as an automatic claim decision.

Introduction

Background

Insurance companies process claims involving information about policyholders, vehicles, accidents, and previous claims. Reviewing this information can require attention to many details. Machine learning can be used to analyze historical claim records and estimate whether a new claim resembles records previously labeled as fraudulent.

This project treats fraud detection as a binary classification task. The model predicts one of two classes: genuine or fraudulent. The web application presents the prediction and estimated fraud probability to the user.

Motivation for the Project

This project was selected to explore how machine learning can be applied to a practical classification problem. It also provides experience with preparing structured data, using an XGBoost model, connecting a model to a Flask application, and displaying results through a web interface.

A further motivation is to understand why accuracy alone may not describe a fraud detection model's performance. Because the dataset contains many more genuine records than fraud-labeled records, measures such as recall and precision are also important.

Overview of the Work Carried Out

The project uses a CSV file named fraud_oracle.csv and a saved model named insurance_fraud_model.json. The Flask application loads these files, displays a claim form, prepares the entered information, and sends it to the model for prediction.

The student's description is that model work was carried out in Google Colab and the application was implemented in Visual Studio Code. The supplied project folder contains the Flask application and saved model, but not the training notebook or code used to produce the model. Therefore, the model-training process cannot be fully verified from the supplied files.

Scope and Responsible-Use Statement

The project focuses on a demonstration of vehicle insurance claim classification. It does not connect to an insurer's live systems, store claims in a database, or make real claim decisions.

A model prediction is an estimate based on patterns in its data. It is not proof that a policyholder committed fraud. A genuine claim could be flagged for review, and a fraud-labeled claim could be missed. A trained person would need to review the claim and supporting evidence before making any decision.

Problem Statement

Case-Study Scenario

Consider a vehicle insurer receiving a submitted claim. The claim contains details such as the vehicle's age and price, the accident area, whether a police report was filed, whether a witness was present, and the policyholder's previous claim history. A reviewer may use these details to decide which claims need further examination.

This project explores whether a machine learning model can provide an initial estimate from selected claim attributes and display that estimate through a web application.

Central Case-Study Question

Can a machine learning model use the available vehicle insurance claim attributes to estimate whether a claim belongs to the fraudulent or genuine class, and present that estimate for human review?

Formal Problem Definition

Let (X) represent the claim attributes and (y) represent the target label:

- (y = 0): genuine claim
- (y = 1): fraud-labeled claim

The classification model estimates a class and a probability for a submitted claim:

[
\hat{y} = f(X), \qquad \hat{p} = P(y=1 \mid X)
]

Here, (\hat{y}) is the predicted class and (\hat{p}) is the model's estimated probability for the fraudulent class. In the application, probability bands are used to display low, review, or high risk. These bands are application thresholds; they do not prove fraud.

Objectives that Guide the Study

1. Examine the vehicle insurance claim dataset and its target labels.
2. Use an XGBoost classifier to estimate the claim class and fraud probability.
3. Compare the reported performance of basic, grid-search, and Bayesian-optimized XGBoost versions.
4. Build a Flask interface for entering claim details and displaying results.
5. Present model predictions separately from rule-based review indicators.
6. Identify limitations that should be addressed before any real-world use.

Assumptions and Limitations

This study assumes that the labels in the supplied dataset are suitable for a classroom classification exercise. It also assumes that the saved model was trained using the same dataset and target definition. The training notebook and evaluation procedure were not included, so those details could not be verified.

The dataset is imbalanced: approximately 6% of records are fraud-labeled. The selected model's reported recall is 36.04%, meaning it identifies only a portion of the fraud-labeled records under the reported evaluation. The web form collects 15 claim attributes, while the dataset has 32 predictor columns. In the current application, predictor columns not supplied by the form are filled with zero. This mismatch may affect predictions.

Some form categories also differ from the dataset's category labels. For example, the dataset uses supplement categories such as 1 to 2 and 3 to 5, while the form uses 1 and 2 to 4. The application maps categories using the dataset's values and fills unknown mappings with zero. This should be corrected by aligning the form choices and feature-preparation pipeline with the model's training data.

The dataset's original source URL, training split, tuning parameters, and detailed confusion-matrix counts are not recorded in the supplied project folder.

Dataset and Exploratory Data Analysis

About the Dataset

The project dataset is fraud_oracle.csv. It contains vehicle insurance claim records.

Dataset property	Observed value
Records	15,420
Total columns	33
Predictor columns	32
Target column	FraudFound_P
Genuine records (0)	14,497
Fraud-labeled records (1)	923
Fraud-labeled share	5.99%
Missing cells	0
Duplicate rows	0

The dataset's original online source link was not present in the supplied project folder and should be added to the final report if available.

Purpose of EDA

Exploratory Data Analysis (EDA) helps describe a dataset before modeling. It can reveal the number of records and columns, missing values, duplicate rows, class balance, and the distribution of individual features. These checks help identify data issues that may affect model training or interpretation.

Preparation Steps Actually Performed

The supplied application code confirms the following steps at prediction time:

1. Read the CSV file and identify all columns other than FraudFound_P as model predictors.
2. Collect claim details from the web form.
3. Convert categorical values using category mappings created from the dataset.
4. Convert numeric values to numeric format.
5. Fill missing or unknown values with zero.
6. Reorder the input columns to match the dataset's predictor-column order.
7. Pass the prepared record to the saved XGBoost model.

The project folder does not contain the training notebook or an EDA script. Therefore, the exact data-cleaning, encoding, train/test split, and tuning steps performed during model development cannot be confirmed from those files.

EDA Exploration – Implementation

The dataset was examined by checking its dimensions, data types, target distribution, missing values, duplicate rows, and selected feature distributions. The following Python commands can be used to reproduce these checks:

```
import pandas as pd


data = pd.read_csv("fraud_oracle.csv")


print(data.shape)
print(data.dtypes.value_counts())
print(data["FraudFound_P"].value_counts())
print(data["FraudFound_P"].value_counts(normalize=True) * 100)
```

```
print(data.isnull().sum().sum())
print(data.duplicated().sum())
print(data["Age"].describe())
```

The checks found 15,420 rows and 33 columns. No missing cells or duplicate rows were found. The target distribution was 14,497 genuine records and 923 fraud-labeled records.

Selected categorical feature counts included:

- Accident area: 13,822 Urban and 1,598 Rural records.
- Police report: 14,992 records with no report and 428 with a report.
- Witness present: 15,333 records with no witness and 87 with a witness.
- Previous claims: 4,352 with none, 3,573 with one, 5,485 with two to four, and 2,010 with more than four.
- Supplements: 7,047 with none, 2,489 with one to two, 2,017 with three to five, and 3,867 with more than five.

These are descriptive counts. They do not show that any one feature causes fraud.

Summary of EDA Findings

The dataset has no missing cells or duplicate rows according to the checks performed. However, its target labels are strongly imbalanced: 94.01% are genuine and 5.99% are fraud-labeled. A model can achieve high overall accuracy while still missing many fraud-labeled cases, so recall and other class-specific measures must also be considered.

The dataset includes categorical claim attributes and numerical values such as age. The form does not collect all dataset predictors, and some category names do not match. These issues should be addressed so the form and model use consistent feature values.

Algorithm Selection and Implementation

The project uses XGBoost, a gradient-boosted decision-tree method suited to structured data. The project's performance page compares three versions: basic XGBoost, grid-search XGBoost, and Bayesian-optimized XGBoost. It identifies the Bayesian-optimized version as the selected model.

Metric	Basic XGBoost	Grid Search	Bayesian XGBoost
Accuracy	95.10%	94.29%	95.43%

Precision	76.14%	92.00%	82.56%
Recall	34.01%	11.68%	36.04%
F1 score	47.02%	20.72%	50.18%
ROC-AUC	66.64%	55.80%	67.76%

The reported Bayesian XGBoost results have the highest accuracy, recall, F1 score, and ROC-AUC among the three listed versions. Grid Search has the highest precision but the lowest recall. The selected model's recall of 36.04% indicates that many fraud-labeled cases may still be missed. Therefore, the accuracy value should not be presented on its own.

The Flask application loads insurance_fraud_model.json and calls the model's prediction and probability functions for each submitted claim. It displays the predicted class and the estimated probability for the fraudulent class. It also calculates rule-based review indicators from selected form fields. These rules are separate from the XGBoost prediction: they can recommend that a claim be reviewed, but they do not change the model's classification.

The performance figures above are transcribed from the project's performance page. Since the training notebook, evaluation split, and confusion matrix were not included, the figures could not be independently reproduced for this report.

Conclusion

This project demonstrates a machine learning workflow for vehicle insurance claim analysis. It uses a labeled claim dataset, a saved XGBoost model, and a Flask web application to present a model classification and estimated fraud probability. It also shows separate rule-based review indicators.

The dataset contains 15,420 records, with a fraud-labeled share of 5.99%. The selected model has a reported accuracy of 95.43%, but its reported recall is 36.04%. This shows why accuracy alone is not enough to judge a fraud detection model. The current form also collects fewer features than the model expects and contains some category labels that differ from the dataset.

Future improvements should align the form and training features, document the dataset source and model-training process, and evaluate the model with a clearly recorded test split and confusion matrix. SafeClaim should remain a learning prototype: its output can support investigation practice, but it should not be used as proof of fraud or as an automatic claim decision

Signature of the Team members